



CUSTOMER DEPLOYMENT / START-TO-FINISH GUIDE

AitlasQM Customer-Owned Installation Manual

Deploy QM, isolated sandboxes, a customer-owned connector gateway, personal app connections, Unified Inbox, and Slack

Production must belong to your company: GitHub, Fly.io, Sprites, model billing, Slack, Composio, gateway, data, and recovery access.

Version 2.0 | Complete connector edition | 11 August 2026

Repository reviewed at main commit 692f5867

1. Read this before you deploy

This guide is for the person who uses the public template at <https://github.com/billysticker/aitlasqm-deploy-starter> to install AitlasQM inside a new company's own infrastructure. It is not an internal ChiroCandy operator guide. Complete the sections in order and do not reuse any production company's values.

OWNERSHIP RULE If a subscription, API key, cloud organization, OAuth app, data store, domain, or recovery email is used in production, it must be owned by your company unless your signed service agreement explicitly says otherwise.

What you are installing

Part	What it does	Who owns it
GitHub repository	Configuration, sandbox tools, brand files, Slack manifests, and deployment history	Your company GitHub organization
Fly.io	Runs the always-on QM core, portal, web UI, admin service, bot, database, and object storage	Your company Fly organization
Sprites	Provides persistent, isolated Linux computers used by agent scopes	Your company through its Fly/Sprites organization
Model provider	Pays for and serves the AI model	Your company OpenRouter or approved provider account
Composio	Stores and manages each user's connected-app authorization	Your company Composio project
Composio gateway	Validates QM identity and exposes approved, read-first connected-app actions	Your company deployment and domain
Slack	Provides team chat, bot access, and web sign-in	Your company Slack workspace

COMPLETE BUT CUSTOMER-OWNED The public starter now includes the generic connector gateway, scoped sandbox client, Connections tab, and Unified Inbox patch source. It still contains no live company URL, credential, Composio project, connected account, private image pin, or company-specific identity/voice skill. Every image, gateway, OAuth configuration, and secret is created in the customer's accounts.

A successful template copy is not proof of a production installation. Final acceptance must include real sign-in, a real model answer, isolated sandbox proof, live OAuth, a harmless connector read, and second-user isolation.

2. The installation map

Use this as the master checklist. The detailed instructions follow in the same order.

Step	Owner action	Done when
1	Create company-owned accounts and choose owners	All accounts use company billing and recovery
2	Write down the customer values in Section 3	No placeholder is blank
3	Install the local tools	Every version check passes
4	Create a private repository from the public template	Your private origin is correct
5	Replace starter placeholders and example content	The customized starter audit passes
6	Install packages and run local checks	QM check, starter audit, and conformance pass
7	Prepare Fly.io and authenticate Sprites	Correct organizations are selected
8	Run guided setup and collect secrets	Local .env is complete and untracked
9	Publish the sandbox and deploy QM	Live check and doctor pass
10	Deploy the included customer connector gateway	Gateway rejects unsigned requests
11	Create Slack sign-in and bot apps	Real portal sign-in and @mention work
12	Run acceptance, isolation, backup, and handoff tests	The owner signs the checklist

EXPECTED EFFORT A first installation should be treated as a controlled deployment, not a five-minute app install. Account approvals, DNS, Slack review, container access, and OAuth configuration may require separate company owners.

Who should be present

- Deployment operator: runs the commands and records evidence.
- Company owner/admin: controls billing, recovery, and cloud membership.
- Slack admin: creates and installs two Slack applications.
- Security or IT owner: approves scopes, domains, connected apps, and secret storage.
- Acceptance tester: uses a normal non-admin account and a second test user.

3. Customer values worksheet

Choose these values once. Use lowercase letters, numbers, and hyphens for slugs. App names must be globally unique where Fly.io requires it.

Value	Example only	Your value
Company display name	Example Dental Group	_____
Company slug / orgId	example-dental	_____
Customer private repository	github.com/exampleco/aitlasqm	_____
Fly organization slug	exampleco	_____
Fly region	iad	_____
Unique app prefix	exampleco-qm	_____
Portal URL	https://exampleco-qm-portal.fly.dev	_____
Sandbox app	exampleco-qm-sandboxes	_____
Sprites organization	exampleco	_____
Model provider	openrouter	_____
Primary admin email	admin@example.com	_____
Reviewer email(s)	owner@example.com	_____
Slack workspace / team ID	T0123456789	_____
Composio project	exampleco-qm	_____
Gateway app / URL	exampleco-qm-gateway	_____
Web UI image app	exampleco-qm-web-ui	_____

NAMING TIP Keep the same recognizable prefix across Fly apps, sandbox app, Composio project, gateway service, and monitoring. This makes ownership and incident response much easier.

Accounts to create before coding

Create company-owned GitHub, Fly.io, Sprites, model-provider, Slack, Composio, and domain/DNS accounts. The gateway runs in the customer's Fly organization. Add at least two company administrators where the provider supports it. Turn on MFA and store recovery codes in the company password manager.

4. Install the required local tools

Tool	Why it is needed	Check
Git	Clone and manage the customer repository	git --version
Node.js 24 + npm	Run the pinned QM CLI	node --version; npm --version
Docker with Buildx	Build and publish sandbox/container images	docker buildx version
Fly CLI (flyctl)	Create, deploy, inspect, and recover Fly services	fly version
OpenSSL	Generate signing and encryption material	openssl version
Sprites CLI	Authenticate the customer sandbox organization	sprite --help
ripgrep (rg)	Find old customer names and unsafe defaults	rg --version

Recommended computer setup

WINDOWS Use WSL 2 with Ubuntu and Docker Desktop's WSL integration. Keep the clone inside the Linux filesystem, such as /home/yourname/work/aitlasqm. Run all repository commands from that same Ubuntu terminal.

On macOS or Linux, use a normal private work folder such as ~/work/aitlasqm. Avoid OneDrive, Dropbox, public web folders, shared drives, and shared family computers because the local setup contains production-capable secrets.

Windows / WSL base setup

```
# Administrator PowerShell
wsl --install
wsl --update

# Then in Ubuntu
sudo apt update
sudo apt install -y git curl ca-certificates openssl build-essential ripgrep
```

Install Node 24 through nvm or the official Node installer. Install Docker Desktop and enable WSL integration. Install flyctl from Fly.io's official installer. Install the Sprites CLI inside the same WSL environment:

```
curl -L https://fly.io/install.sh | sh
curl -fsSL https://sprites.dev/install.sh | sh

node --version
npm --version
docker buildx version
fly version
sprite --help
openssl version
rg --version
```

VERSION GATE Do not continue until Node reports version 24 or newer and Docker Buildx works. A later deploy failure is harder to diagnose than a failed tool check here.

5. Create your repository from the public starter

The class share link is <https://github.com/bilysticker/aitlasqm-deploy-starter>. The destination must be a private repository owned by your company. Do not request or use access to a live deployment repository.

[Open the public AitlasQM deploy starter](#)

Path A - GitHub website (recommended)

Open the starter while signed in to GitHub. Select Use this template, then Create a new repository. Choose the customer-owned account or organization, give the repository a customer-specific name, set visibility to Private, and create it. Clone that new private repository:

```
mkdir -p ~/work
cd ~/work
git clone <customer-repo-url> aitlasqm
cd aitlasqm
git remote -v
git status
```

Path B - GitHub CLI

An authenticated GitHub CLI can create and clone the private customer repository directly from the same public template:

```
cd ~/work
gh repo create <customer-owner>/<customer-repo> --private --template bilysticker/aitlasqm-
deploy-starter --clone
cd <customer-repo>
git remote -v
git status
```

BEFORE THE FIRST PUSH Do not copy local .env files, token files, cloud credentials, or generated provider state. Confirm .gitignore covers them and run the secret check in Section 7.

Create an installation branch

```
git switch -c setup/<company-slug>-customer-deployment
```

Make and review the customerization changes on this branch. Do not deploy while starter placeholders remain.

6. Replace the starter placeholders

Open `qm.config.jsonc` first. Replace the safe example values before running any command that creates cloud resources.

Setting	Replace with	Why
orgId	<company-slug>	Identity and scope namespace
publicUrl	Your final HTTPS portal URL	Redirects, health checks, and sign-in
appPrefix	Globally unique customer prefix	Names the Fly applications
region	Approved Fly region	Data location and latency
flyOrg	Customer Fly organization slug	Billing and ownership boundary
sandbox.app	Customer sandbox app name	Image and runtime boundary
modelProvider	Customer-approved provider	Model credentials and billing
portal allowed emails/team	Customer admins and Slack team ID	Prevents cross-workspace access
S3_BUCKET	Customer-owned bucket name	Snapshot and transfer storage boundary
Gateway URL	Customer gateway <code>/api/qm/composio</code> URL	Connections and sandbox egress
Reviewer emails	Customer connection/memory reviewers	Routes requests to accountable humans

Publish only customer-owned images

The public starter ships the reviewed web UI source patches and sandbox layer, but no deployed image pin. Each customer builds and publishes both images into its own Fly registry.

- Run the customized starter audit before any cloud image publish.
- Use `npm run web-ui:publish` to build, push, resolve, and pin the customer web UI image.
- Run `qm sandbox publish` only in the verified customer Fly organization.
- Record and review the customer-generated sandbox digest before `qm up`.

STOP GATE Do not run `qm up` while `replace-me`, `admin@example.com`, `T00000000000`, or `ExampleCo` remains in the deployment config or Slack manifests. Do not add an image, domain, or connector endpoint that the customer does not own.

7. Customize and audit the customer copy

The public repository includes only fake names and a generic greeting example. Replace or remove those examples before the first customer deployment.

Required cleanup

- Rename the package in package.json to the customer's private deployment name.
- Update slack-app-manifest.yml and slack-sso-manifest.yml with the customer app name, description, and final callback URL.
- Replace or delete sandbox/skills/greet and sandbox/tools/example-tool after using them to understand the sandbox shape.
- Add only customer-approved identity, voice, knowledge, and connector skills.
- Keep the included connector-gateway customer-owned; never move its secrets into QM, a Sprite, or git.

Run the starter gate

```
npm run starter:audit -- --require-customized
rg -n 'replace-me|admin@example\.com|T0000000000|ExampleCo' qm.config.jsonc
sandbox/tools/composio/tool.json slack-*.yaml
```

The audit must pass and the explicit placeholder scan must return no matches. Review the complete diff as well; automated patterns do not replace a human ownership and scope review.

STARTER PROTECTION The audit blocks common credential patterns and known live-production topology, requires the web and sandbox gateway hosts to match the tool egress allowlist, and rejects accidentally present .env files. The final complete mode also requires a customer-built web UI image pin.

Review and commit the clean baseline

```
git status
git diff --check
git diff
git add <reviewed-files>
git commit -m "chore: prepare customer-owned AitlasQM deployment"
```

8. Install and validate locally

These checks do not deploy anything. They verify the customer copy and the pinned QM CLI before cloud setup.

1. Install exactly what both lock files specify.

Install the QM overlay and the separate gateway dependencies reproducibly.

```
npm ci
npm run gateway:install
```

2. Confirm the pinned QM CLI.

The reviewed repository pins @yc-software/qm 0.1.12. If the delivered commit changes, record the new version.

```
npm list @yc-software/qm --depth=0
```

3. Run the offline configuration check.

Fix all reported errors before continuing.

```
npm exec qm -- check
```

4. Run connector, gateway, and policy checks.

These validate scope policy, the sandbox client, and the gateway production build without proving live OAuth.

```
npm run starter:audit -- --require-customized
npm run conformance
npm run test:composio
npm run gateway:test
npm run gateway:build
npm --prefix connector-gateway audit --omit=dev
```

5. Check for accidentally tracked secrets.

This is a name/pattern check, not a substitute for a dedicated secret scanner.

```
git status --short
git ls-files | rg '^(^|/)(\.env|\.token\.|\.secret\.|\.credential\.)*$'
```

EXPECTED RESULT All tests/builds pass, the gateway production audit reports zero vulnerabilities, git status shows only intentional customerization, and no credential file is tracked. Local green still does not prove Fly, Slack, the model, OAuth, or user isolation.

9. Prepare Fly.io and Sprites

Fly.io account and organization

1. Sign in with the company-owned Fly account.

Use a named human account for setup. Do not share one person's password.

```
fly auth login
fly auth whoami
fly orgs list
docker buildx version
```

2. Confirm the selected organization.

The slug must exactly match `qm.config.jsonc flyOrg`. Confirm company billing and at least two administrators in the Fly dashboard.

3. Create the customer image and gateway apps.

Create only after the org, global names, and expected billing are verified.

```
fly apps create <sandbox-app> --org <fly-org>
fly apps create <app-prefix>-web-ui --org <fly-org>
fly apps create <gateway-app> --org <fly-org>
```

Sprites authentication

Sprites uses the Fly.io identity. Authenticate the organization that will own the persistent agent computers:

```
sprite org auth --org <sprites-org>
sprite org list
```

DO NOT CREATE ONE MANUALLY Do not manually create a Sprite for every employee. QM creates and manages isolated agent computers according to scope. Your job is to authorize the correct organization and later prove that two users do not share private state.

Use limited deployment credentials

For automation, prefer a narrow Fly organization or app-scoped token rather than a general personal authentication token. Store it only in the approved secret manager or CI/CD secret store. The guided setup can collect it without placing it in Git.

10. Run guided setup and collect secrets

Use QM's setup command as the main secret-entry workflow. It reads `qm.config.jsonc` and creates the local `.env` contract. Never paste real values into this manual, a ticket, Slack, or GitHub.

```
npm exec qm -- setup .
```

What you will need

Secret group	Owner	Purpose
Fly deployment credential	Company cloud admin	Create and update the customer's Fly applications
Sprites token / authorization	Company sandbox admin	Allow QM to create isolated persistent computers
Model API key	Company AI billing owner	Pay for live model responses
QM signing/encryption values	Deployment operator	Protect sessions, scopes, and internal messages
Initial admin identity	Company owner	Bootstrap controlled access
Slack client and bot secrets	Slack admin	Added after the two Slack apps are created
Public API key, if enabled	Security owner	Controlled API entry point

CONNECTOR SECRET BOUNDARY Do not put the Composio project API key or gateway identity secret in QM, the root `.env`, or a user's Sprite. They belong only in the included connector-gateway service's customer Fly secrets. Users connect through short-lived Connect Links bound to their QM scope.

Protect the local secret file

```
chmod 600 .env
git check-ignore .env
git status --short
```

If `git check-ignore` does not print `.env`, stop and fix `.gitignore` before entering any real value. Back up secrets to the company password manager or approved secret vault, not to a second plaintext file.

11. Publish the web UI and sandbox, then deploy QM

Run this sequence from the customer repository, in the same terminal where Fly, Docker, and Sprites authentication were verified.

1. Build and pin the customer web UI.

The script applies the reviewed upstream patches, pushes to the customer Fly registry, resolves the digest, and updates qm.config.jsonc.

```
npm run web-ui:prepare
npm run web-ui:publish -- --app <app-prefix>-web-ui
git diff -- qm.config.jsonc
```

2. Run the complete tracked-file gate.

This requires the immutable customer web UI pin and matched gateway egress.

```
npm run starter:audit -- --require-complete
npm exec qm -- check
```

3. Render Slack and publish the sandbox.

The sandbox image contains the scoped Composio client but no Composio API key.

```
npm exec qm -- slack render
npm exec qm -- sandbox publish
```

4. Push approved QM secrets.

Only names and destinations should appear in logs; never paste values into deployment notes.

```
npm exec qm -- secrets push
```

5. Review the plan before mutation.

Confirm the Fly organization, app prefix, region, portal URL, database, storage, and both image registries belong to the customer.

```
npm exec qm -- plan
```

6. Deploy core QM.

QM creates or reuses the configured services. Private Postgres and object storage may be provisioned automatically when customer credentials are not supplied.

```
npm exec qm -- up --yes
```

7. Verify the live core deployment.

Run diagnostics and save the output before deploying the gateway.

```
npm exec qm -- doctor
npm exec qm -- check --live
```

EXPECTED RESULT The portal and service health checks succeed in the customer Fly organization. Run qm up a second time; it should converge safely without creating duplicate infrastructure.

12. Understand the included connected-app path

INCLUDED IN THIS STARTER The repository now contains connector-gateway/, the scoped sandbox Composio client and skills, reproducible AitlasQM web UI patches, the Connections tab, and Unified Inbox. Students still create their own Composio project, gateway app, OAuth configurations, image, and connected accounts.

Each customer needs one protected project and gateway, then a separate connected account per person and app. The gateway validates QM's short-lived capability before deriving a non-reversible identity for the exact personal or approved private scope.

```

TEAM MEMBER (stable QM identity)
|
+--> QM personal scope / DM
    |
    +--> isolated Sprite
        |
        +--> signed request to customer gateway
            |
            +--> customer Composio project
                |
                +--> that user's Gmail / Drive / Fathom account
  
```

Question	Answer
Does every user need a Composio login?	Usually no. Each user opens the generated Connect Link and completes the app's OAuth or credential screen.
Does every user share one Gmail connection?	No. Personal connections are tied to a stable user identity and must remain private.
Does an admin create a Sprite for every user?	No. QM manages isolated agent computers based on scope.
Where is the Composio API key?	Only in the protected customer gateway service.
Can the agent send or modify data?	Sandbox tools are read-only. Unified Inbox sends require a signed-in personal user, exact confirmation, and a unique request ID.
What is shared?	Only an admin-approved organization connection in the designated private shared scope.

ISOLATION RULE Start personal app connections from a direct message or personal chat. Never create a person's Gmail, Drive, calendar, or meeting connection inside a shared company channel.

13. Create the connector project and deploy the gateway

A. Create the Composio project

1. Create a dedicated project.

Use <company-slug>-qm. A project is the isolation boundary for API keys, auth configurations, connected accounts, and webhooks.

2. Create a project API key.

In the Composio dashboard, open Project Settings, then API Keys. Copy it directly into the customer gateway secret manager.

3. Create auth configurations.

Enable only the apps approved for launch. For OAuth apps, choose managed OAuth or enter the customer's own OAuth client. For API-key apps such as some Fathom setups, create the matching API_KEY auth configuration.

4. Record the auth configuration IDs.

Use toolkit names as the map keys, for example fathom to ac_.... Do not record the API key in the worksheet.

B. Deploy the included customer gateway

The implementation is in connector-gateway/. Its package is independent from QM and deploys into the customer's Fly app. Configure these values only as gateway Fly secrets:

```
GATEWAY_PUBLIC_URL=https://<gateway-app>.fly.dev
QM_COMPOSIO_API_KEY=<customer-composio-project-api-key>
QM_COMPOSIO_IDENTITY_SECRET=<random-32-byte-secret>
QM_CORE_API_URL=https://<app-prefix>-core.fly.dev
QM_COMPOSIO_AUTH_CONFIGS_JSON={"fathom":"ac_<customer-id>"}
```

Generate the identity secret in the operator terminal and enter it directly into the gateway secret manager:

```
openssl rand -hex 32
cd connector-gateway
npm ci
npm test
npm run build
fly deploy --app <gateway-app>
curl https://<gateway-app>.fly.dev/healthz
cd ..
```

SECURITY TEST The health response must report configured true. Send a request to /api/qm/composio/connections without the QM capability header; it must return 401. If it accepts unsigned or unscoped traffic, stop.

C. Point QM at the gateway

The customer gateway URL was already set in qm.config.jsonc and the sandbox egress descriptor. After the live security test, republish and redeploy so every surface uses the verified gateway:

```
npm run gateway:test
npm exec qm -- sandbox publish
npm exec qm -- plan
npm exec qm -- up --yes
npm exec qm -- check --live
```

14. Connect each teammate's apps

Repeat this short process for each teammate and each approved personal app. The connection is created only after that person completes the provider's authorization screen.

1. Open a personal scope.

Use the teammate's QM direct message or personal web chat, signed in as that person.

2. Open Connections or ask QM to connect.

Use Add your own connections in the web UI, or say Connect my Gmail account. The scoped command is `composio connect <toolkit>`.

3. Open the Connect Link.

The teammate follows the link and completes Google, Microsoft, Fathom, or other provider authorization. They should confirm the displayed app, scopes, and customer name before approving.

4. Confirm active status.

Return to the same personal scope and ask QM to list connections. The connection must show active for that user.

5. Run a harmless read test.

Search or read a non-sensitive test item. For Gmail or Fathom sync, verify only compact summaries, action items, and source pointers are stored as designed.

```
composio connections
composio search "<safe-query>" --toolkit <toolkit>
composio read <TOOL_SLUG> --json '{}'
```

Prove user isolation

1. Test as User A.

Connect User A's test account and confirm a safe item can be read.

2. Sign in as User B.

Open User B's personal scope. User A's connection and data must not appear.

3. Test shared scope separately.

Only an organization admin may connect an approved shared account in the designated private company channel.

4. Test Unified Inbox separately.

Confirm User A sees only User A's connected sources. If sends are enabled, use a controlled recipient and require exact signed-in confirmation.

PRODUCTION ACCEPTANCE A working Connect Link is not enough. Record OAuth completion, active status, a safe read, correct Inbox scope, and proof that User B cannot access User A's connection or messages.

15. Set up Slack: two applications

AitlasQM uses two separate Slack apps. Do not combine them: one app is the bot used in channels and DMs; the other is the OpenID Connect app used for portal sign-in.

Slack app	Purpose	Key proof
Bot app	Receives mentions and direct messages	A normal user can @mention it and receive a real model response
SSO app	Signs approved workspace members into the web portal	A normal user completes Slack sign-in and reaches the portal

1. Render the customer manifests.

Generate links and review the customer name, descriptions, permissions, redirect URLs, event subscriptions, and workspace boundary.

```
npm exec qm -- slack render
npm exec qm -- outputs
```

2. Create the SSO app.

Use the rendered SSO manifest. Copy the client ID and client secret into the approved secret flow, then redeploy.

3. Create the bot app.

Use the rendered bot manifest, install it to the customer workspace, and store the bot credentials through QM Admin or the approved secret path.

4. Invite and test.

Invite the bot into a private test channel and send a real @mention from a normal user. Also test a direct message.

WORKSPACE BOUNDARY The allowed Slack team ID and the installed workspace must match. A healthy portal URL does not prove Slack sign-in or bot events work.

16. Final acceptance checklist

The installation is ready only after every required item below has named evidence and an owner. Marking a command green is not the same as proving the user journey.

Test	Pass condition	Evidence / owner
Ownership	All production accounts, billing, recovery, and domains are customer controlled	_____
Starter audit	No placeholder, foreign production value, credential file, or unowned image remains	_____
Local validation	QM, connector client, gateway tests/build, audit, and conformance pass	_____
Web UI image	Customer registry digest is pinned; Connections and Unified Inbox load	_____
Idempotence	A second qm up creates no duplicate infrastructure	_____
Portal sign-in	Normal customer user signs in through customer Slack	_____
Real model	User receives a real provider-backed response	_____
Sandbox isolation	Agent creates a random proof file; operator independently verifies the correct scoped computer	_____
Connector user A	OAuth completes, connection is active, and safe read succeeds	_____
Connector user B	Cannot see or use User A's connection, Inbox, or knowledge	_____
Shared connection	Admin-approved connection is visible only in designated private shared scope	_____
Inbox send gate	Controlled send requires exact signed-in confirmation and is not duplicated	_____
Slack bot	Normal user gets a real response to @mention and DM	_____
Recovery	Previous sandbox digest and rollback command are recorded	_____
Monitoring	Owner knows where to inspect status, logs, spend, and alerts	_____

Sandbox proof pattern

Ask the agent in User A's scope to create a UUID in /root/workspace/qm-computer-proof.txt. The deployment operator then locates the Fly Machine selected for that agent_scope and reads the file independently. Repeat for User B and confirm the file and scope do not cross.

OWNER SIGN-OFF Record the deployment commit, QM version, sandbox and web UI digests, gateway release, acceptance date, evidence location, and approving customer owner.

17. Routine operation, rollback, and troubleshooting

Safe routine commands

```
npm exec qm -- check --live
npm exec qm -- doctor
npm exec qm -- outputs
npm exec qm -- plan
```

Use the Fly dashboard and flyctl for service status and logs. Use the provider dashboards for model spend, Composio connections, Slack installation, database health, object storage, and gateway alerts.

Rollback

Record the previous sandbox digest before every sandbox publish. If a sandbox release fails acceptance, use the exact verified digest:

```
npm exec qm -- rollback --to <previous-sandbox-digest>
```

If the connector layer is unsafe, disable the affected gateway routes or auth configuration before retrying. Do not delete data stores during incident response. The qm down command stops services but does not authorize database, bucket, volume, connected-account, or audit-log deletion.

Common problems

Symptom	Check first
Fly app created in the wrong account	Stop. Verify fly auth whoami, fly orgs list, and qm.config.jsonc flyOrg.
qm up reports no sandbox image	Run qm sandbox publish in the verified customer Fly organization, then review the recorded digest.
Gateway reports host not allowed	Use the customer-specific security-reviewed hostname allowlist; do not bypass validation.
Composio Connect Link fails	Check customer project, auth config ID, toolkit approval, redirect URI, and gateway logs.
One user sees another's connection	Disable the connector route immediately and audit stable identity and scope mapping.
Portal health is green but login fails	Check the separate Slack SSO app, redirect URLs, client secret, and allowed team ID.
Slack bot is silent	Check installation, bot token, event subscription URL, app invitation, scopes, and actual bot username.
Model answer fails	Check provider key, billing/quota, configured model, and core service logs.

18. Handoff record and official references

Customer handoff record

Item	Record
Customer repository and default branch	_____
Deployment commit	_____
QM CLI version	_____
Fly organization and applications	_____
Sprites organization	_____
Composio project and approved toolkits	_____
Gateway service, domain, and release	_____
Web UI image and immutable digest	_____
Slack SSO app and bot app owners	_____
Secret manager / recovery location	_____
Monitoring and incident owner	_____
Acceptance evidence location	_____
Customer approver and date	_____

Repository references

- README.md — repository entry point and local validation commands.
- deployment.md — owner/operator deployment and acceptance runbook.
- qm.config.jsonc — actual deployment contract; review before every plan or up.
- .env.example — supported secret names; never add real values here.
- CUSTOMER-WORKSHEET.md — customer ownership, access, provider, and acceptance decisions.
- RELEASE-CHECKLIST.md and SECURITY.md — publication and incident-response gates.
- connector-gateway/README.md — customer gateway boundary, checks, and Fly deployment.
- patches/ and scripts/prepare-qm-web-ui.mjs — reproducible Connections and Unified Inbox source.

Official provider documentation

Use the repository files for this deployment's exact contract and these provider pages for current account, authentication, and platform instructions.

[Fly.io apps](#) — official provider documentation

[Fly.io access tokens](#) — official provider documentation

[Sprites quickstart](#) — official provider documentation

[Sprites authentication](#) — official provider documentation

[Composio projects](#) — official provider documentation

[Composio API keys](#) — official provider documentation

[Composio auth configurations](#) — official provider documentation

[Composio connected accounts](#) — official provider documentation

[Composio user authentication](#) — official provider documentation

LAST CHECK Provider interfaces and commands change. Before a new deployment, compare this manual with the delivered repository commit and the linked official provider documentation.

September 2026 template refresh

The public template now pins @yc-software/qm 0.1.12 and applies its AitlasQM web UI changes to the reviewed upstream revision. Use the current README and deployment.md for commands; do not reuse an image built from the older patch series.

Specialist rooms

Create private Slack channels or QM room contexts for each mission. The room-skills directory contains editable image-design and landing-page examples. These are not installed automatically. Review each room's members, tools, model access, credentials, and human approval rules before publishing its skill through the QM Skills editor.

Company Brain

QM includes scoped memory. The Brain Map interface can be connected to a separate, customer-owned knowledge service only after its identity, source grants, ingestion consent, and isolation checks are configured. This template does not deploy the ChiroCandy Brain service. Read docs/COMPANY-BRAIN-ADOPTION.md before enabling a knowledge integration.

Customer branding

Set the customer's final botName and QM_BRAND_MARK_URL for web UI and admin. The starter placeholders and the included AitlasQM logo are examples. Never copy another company's production URL, private image digest, staff email, Slack team ID, or provider secret into a customer repository.